STAT4064 Assignment 2

Jason Wang - 24839132

Today

Contents

1 Question 1 **2**

1.1 (a) . . . 2

1.2 (b) . . . 2

1.3 (c) . . . 3

1.4 (d) . . . 4

1.5 (e) . . . 5

2 Question 2 **5**

2.1 (a) . . . 5

2.2 (b) . . . 6

2.3 (c) . . . 6

2.4 (d) . . . 7

2.5 (e) . . . 8

3 Question 3 **11**

3.1 (a) . . . 11

3.2 (b) . . . 12

3.3 (c) . . . 12

4 Question 4 **13**

4.1 (a) . . . 13

4.2 (b) . . . 14

4.3 (c) . . . 14

4.4 (d) . . . 18

4.5 (e) . . . 19

Assignment 2 submitted by Jason Wang, 24839132

1 Question 1

1.1 (a)

```
library(datasets)
head(iris, 8)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1          5.1         3.5          1.4         0.2  setosa
## 2          4.9         3.0          1.4         0.2  setosa
## 3          4.7         3.2          1.3         0.2  setosa
## 4          4.6         3.1          1.5         0.2  setosa
## 5          5.0         3.6          1.4         0.2  setosa
## 6          5.4         3.9          1.7         0.4  setosa
## 7          4.6         3.4          1.4         0.3  setosa
## 8          5.0         3.4          1.5         0.2  setosa
```

We first import the dataset by calling datasets library and using head to show the first 8 rows of the dataset. We see the iris dataset contains 5 variables, these are Sepal.Length, Sepal.Width, Petal.Length, Petal.Width and Species of the flower.

1.2 (b)

The Posterior Probability is given as:

$$P(Y = k \mid X = x) = \frac{\pi_k f_k(x)}{\sum_{l=1}^{2} \pi_l f_l(x)}$$

Where:

$$f_k(x) = \frac{1}{\sqrt{2}\pi\sigma_k} e^{-\frac{1}{2}\left(\frac{x-\mu_k}{\sigma_k}\right)^2}$$

Under LDA, we assume common variance. We can rewrite and simplify the posterior probability by rewriting it to:

$$p_k(x) = \frac{\pi_k e^{(-\frac{x-\mu_k}{2\sigma})^2}}{\sum_{l=1}^{K} \pi_l e^{(-\frac{x-\mu_l}{2\sigma})}}$$

To write the formula for each class, we first find the mean and variance for each class then plug it into the posterior formula.

```
#unique(iris$Species)
setosa_n <- sum(iris$Species == 'setosa')
versicolor_n <- sum(iris$Species == 'versicolor')
virginica_n <- sum(iris$Species == 'virginica')

setosa_length <- iris$Petal.Length[iris$Species == 'setosa']
versicolor_length <- iris$Petal.Length[iris$Species == 'versicolor']
virginica_length <- iris$Petal.Length[iris$Species == 'virginica']

mu_setosa <- mean(setosa_length)
var_setosa <- var(setosa_length)
```

```
mu_versicolor <- mean(versicolor_length)
var_versicolor <- var(versicolor_length)
mu_virginica <- mean(virginica_length)
var_virginica <- var(virginica_length)

var <- ((setosa_n-1)*var_setosa + (versicolor_n-1)*var_versicolor + (virginica_n-1)*var_virginica) /((s
```

We can now plot the formula for each species: The formula for the posterior probability for setosa species is:

$$P(Y = setosa \mid X = x) = \frac{\pi_{setosa} e^{-(\frac{x-1.462}{2\times 0.1852})^2}}{\pi_{setosa} e^{-(\frac{x-1.462}{2\times 0.1852})^2} + \pi_{versicolor} e^{-(\frac{x-4.26}{2\times 0.1852})^2} + \pi_{virginica} e^{-(\frac{x-5.552}{2\times 0.1852})^2}}$$

The formula for the posterior probability for versicolor species is:

$$P(Y = versicolor \mid X = x) = \frac{\pi_{versicolor} e^{-(\frac{x-4.26}{2\times 0.1852})^2}}{\pi_{setosa} e^{-(\frac{x-1.462}{2\times 0.1852})^2} + \pi_{versicolor} e^{-(\frac{x-4.26}{2\times 0.1852})^2} + \pi_{virginica} e^{-(\frac{x-5.552}{2\times 0.1852})^2}}$$

The formula for the posterior probability for virginica species is:

$$P(Y = virginica \mid X = x) = \frac{\pi_{virginica} e^{-(\frac{x-5.552}{2\times 0.1852})^2}}{\pi_{setosa} e^{-(\frac{x-1.462}{2\times 0.1852})^2} + \pi_{versicolor} e^{-(\frac{x-4.26}{2\times 0.1852})^2} + \pi_{virginica} e^{-(\frac{x-5.552}{2\times 0.1852})^2}}$$

1.3 (c)

```
x_new = 2.4
sigma <- sqrt(var)

f_setosa <- dnorm(x_new, mu_setosa, sigma)
f_versicolor <- dnorm(x_new, mu_versicolor, sigma)
f_virginica <- dnorm(x_new, mu_virginica, sigma)

pi = 1/3
num_setosa <- pi * f_setosa
num_versicolor <- pi * f_versicolor
num_virginica <- pi * f_virginica

denominator <- num_setosa + num_versicolor + num_virginica

post_setosa <- round(num_setosa/denominator,4)
post_versicolor <- round(num_versicolor/denominator, 4)
post_virginica <- round(num_virginica/denominator, 4)

post_setosa
```

```
## [1] 0.9991
```

```
post_versicolor
```

```
## [1] 9e-04
```

```
post_virginica
```

```
## [1] 0
```

The results show the probability that a flower with 2.4cm Petal.Length may be classified as. The first one shows that given a flower with 2.4cm Petal.Length, there is a 0.9991 probability that it is a setosa species flower, a 0.0009 probability that is is a versicolor species flower, and lastly a 2.4708e-11 probability that it is a virginica species flower. Given that setosa has the highest posterior probability, the LDA will classify this particular flower as setosa.

1.4 (d)

For QDA, the difference is that we do not use pooled variance and instead use individual variance of each class.Thus the posterior probability formula is:

$$p_k(x) = \frac{\pi_k \frac{1}{\sqrt{2}\pi\sigma} e^{-\frac{1}{2}(\frac{x-\mu_k}{\sigma})^2}}{\sum_{l=1}^{K} \pi_l \frac{1}{\sqrt{2}\pi\sigma} e^{-\frac{1}{2}(\frac{x-\mu_l}{\sigma})^2}}$$

QDA posterior probability formula for setosa is:

$$P(Y = setosa \mid X = x) = \frac{\pi_{setosa} \frac{1}{\sqrt{2}\pi\sigma_{setosa}} e^{-\frac{1}{2}(\frac{x-\mu_{setosa}}{\sigma_{setosa}})^2}}{\pi_{setosa} \frac{1}{\sqrt{2}\pi\sigma_{setosa}} e^{-\frac{1}{2}(\frac{x-\mu_{setosa}}{\sigma_{setosa}})^2} + \pi_{versicolor} \frac{1}{\sqrt{2}\pi\sigma_{versicolor}} e^{-\frac{1}{2}(\frac{x-\mu_{versicolor}}{\sigma_{versicolor}})^2} + \pi_{virginica} \frac{1}{\sqrt{2}\pi\sigma_{virginica}} e^{-\frac{1}{2}(\frac{x-\mu_{virginica}}{\sigma_{virginica}})^2}}$$

QDA posterior probability formula for versicolor is:

$$P(Y = versicolor \mid X = x) = \frac{\pi_{versicolor} \frac{1}{\sqrt{2}\pi\sigma_{versicolor}} e^{-\frac{1}{2}(\frac{x-\mu_{versicolor}}{\sigma_{versicolor}})^2}}{\pi_{setosa} \frac{1}{\sqrt{2}\pi\sigma_{setosa}} e^{-\frac{1}{2}(\frac{x-\mu_{setosa}}{\sigma_{setosa}})^2} + \pi_{versicolor} \frac{1}{\sqrt{2}\pi\sigma_{versicolor}} e^{-\frac{1}{2}(\frac{x-\mu_{versicolor}}{\sigma_{versicolor}})^2} + \pi_{virginica} \frac{1}{\sqrt{2}\pi\sigma_{virginica}} e^{-\frac{1}{2}(\frac{x-\mu_{virginica}}{\sigma_{virginica}})^2}}$$

QDA posterior probability formula for virginica is:

$$P(Y = virginica \mid X = x) = \frac{\pi_{virginica} \frac{1}{\sqrt{2}\pi\sigma_{virginica}} e^{-\frac{1}{2}(\frac{x-\mu_{virginica}}{\sigma_{virginica}})^2}}{\pi_{setosa} \frac{1}{\sqrt{2}\pi\sigma_{setosa}} e^{-\frac{1}{2}(\frac{x-\mu_{setosa}}{\sigma_{setosa}})^2} + \pi_{versicolor} \frac{1}{\sqrt{2}\pi\sigma_{versicolor}} e^{-\frac{1}{2}(\frac{x-\mu_{versicolor}}{\sigma_{versicolor}})^2} + \pi_{virginica} \frac{1}{\sqrt{2}\pi\sigma_{virginica}} e^{-\frac{1}{2}(\frac{x-\mu_{virginica}}{\sigma_{virginica}})^2}}$$

To calculate the posterior probability, we need to calculate the individual sigma by square rooting each individual variance. The calculations for the posterior probability are shown below.

```
x_new = 2.4
sigma_setosa <- sqrt(var_setosa)
sigma_versicolor <- sqrt(var_versicolor)
sigma_virginica <- sqrt(var_virginica)

f_setosa <- dnorm(x_new, mu_setosa, sigma_setosa)
f_versicolor <- dnorm(x_new, mu_versicolor, sigma_versicolor)
f_virginica <- dnorm(x_new, mu_virginica, sigma_virginica)

pi = 1/3
num_setosa <- pi * f_setosa
num_versicolor <- pi * f_versicolor
num_virginica <- pi * f_virginica
```

```r
denominator <- num_setosa + num_versicolor + num_virginica

post_setosa <- round(num_setosa/denominator, 4)
post_versicolor <- round(num_versicolor/denominator, 4)
post_virginica <- round(num_virginica/denominator, 4)

post_setosa
```

```
## [1] 0.0031
```

```r
post_versicolor
```

```
## [1] 0.9967
```

```r
post_virginica
```

```
## [1] 2e-04
```

Given that QDA models each class' variance, given a 2.4 cm Petal.Length flower, QDA predicts that there is a 0.0031 probability of being classified as setosa, a 0.9967 probability of being classified as versicolor, and a 0.0002 probability as virginica.Given versicolor has the highest probability, this means QDA classifies a 2.4 cm Petal.Length as versicolor species.

1.5 (e)

Yes, there is a difference in prediction. For LDA it predicts that a 2.4 cm Petal.Length flower is setosa, while QDA classifies it as versicolor. The most likely reason is that LDA assumes linear separation, where the classes share a common variance, while QDA assumes each class' own specific variance when modeling the posterior probability. LDA is less flexible because it assumes all classes have the same covariance, while QDA is more flexible when the boundary is non-linear. Given the different variance of each species, QDA is a more appropriate model for capturing true class distributions.

2 Question 2

2.1 (a)

We first read in the winequality-red.csv in as Wine, and we check the structure of the data set. We see quality is in integers, we will first turn them into factors using as.factor() and then check all the levels of the quality variable.

```r
Wine <- read.csv('winequality-red.csv', sep = ';')
str(Wine)
```

```
## 'data.frame':    1599 obs. of  12 variables:
##  $ fixed.acidity       : num  7.4 7.8 7.8 11.2 7.4 7.4 7.9 7.3 7.8 7.5 ...
##  $ volatile.acidity    : num  0.7 0.88 0.76 0.28 0.7 0.66 0.6 0.65 0.58 0.5 ...
##  $ citric.acid         : num  0 0 0.04 0.56 0 0 0.06 0 0.02 0.36 ...
##  $ residual.sugar      : num  1.9 2.6 2.3 1.9 1.9 1.8 1.6 1.2 2 6.1 ...
```

```
##  $ chlorides           : num  0.076 0.098 0.092 0.075 0.076 0.075 0.069 0.065 0.073 0.071 ...
##  $ free.sulfur.dioxide : num  11 25 15 17 11 13 15 15 9 17 ...
##  $ total.sulfur.dioxide: num  34 67 54 60 34 40 59 21 18 102 ...
##  $ density             : num  0.998 0.997 0.997 0.998 0.998 ...
##  $ pH                  : num  3.51 3.2 3.26 3.16 3.51 3.51 3.3 3.39 3.36 3.35 ...
##  $ sulphates           : num  0.56 0.68 0.65 0.58 0.56 0.56 0.46 0.47 0.57 0.8 ...
##  $ alcohol             : num  9.4 9.8 9.8 9.8 9.4 9.4 9.4 10 9.5 10.5 ...
##  $ quality             : int  5 5 5 6 5 5 5 7 7 5 ...
```

```
Wine$quality <- as.factor(Wine$quality)
levels(Wine$quality)
```

```
## [1] "3" "4" "5" "6" "7" "8"
```

The levels of wine quality is from 3 to 8. These score the quality of the wine.

2.2 (b)

We first import the MASS library in order to use the lda() function. Then we calculate the predictions and then we can get the classification error for the LDA model.

```
library(MASS)
```

```
## Warning: package 'MASS' was built under R version 4.4.3
```

```
wine_lda <- lda(quality ~ ., data = Wine)
wine_pred <- predict(wine_lda, Wine)
lda_class_err <- mean(wine_pred$class != Wine$quality)
round(lda_class_err, 4)
```

```
## [1] 0.3965
```

The LDA classifier model has achieved a classification error of 0.3965, meaning it misclassifies around 39.65% of the wine samples. This suggests that the wine data overlaps and are not separable by a linear boundary.

2.3 (c)

We first import the nnet library in order to use the multinom() function. Then we calculate the predictions and then we can get the classification error for the multinomial model.

```
library(nnet)
wine_multi <- multinom(quality ~ ., data = Wine, maxit = 1000)
```

```
## # weights:  78 (60 variable)
## initial  value 2865.023391
## iter  10 value 1963.486425
## iter  20 value 1795.520694
## iter  30 value 1541.529008
## iter  40 value 1479.891422
```

```
## iter  50 value 1471.286160
## iter  60 value 1469.045606
## iter  70 value 1466.890589
## iter  80 value 1466.111779
## iter  90 value 1465.874531
## iter 100 value 1465.552983
## iter 110 value 1465.371954
## iter 120 value 1465.163918
## iter 130 value 1465.064718
## iter 140 value 1465.052367
## iter 150 value 1465.033690
## iter 160 value 1464.908693
## iter 170 value 1464.694613
## iter 180 value 1464.511253
## iter 190 value 1464.266836
## iter 200 value 1464.066525
## iter 210 value 1463.769756
## iter 220 value 1462.994644
## iter 230 value 1461.889203
## iter 240 value 1460.824287
## final  value 1460.811130
## converged
```

```r
wine_multi_pred <- predict(wine_multi, data = Wine)
multinom_class_err <- mean(wine_multi_pred != Wine$quality)
round(multinom_class_err, 4)
```

```
## [1] 0.3934
```

The multinomial classifier model has achieved a classification error of 0.3934, meaning it misclassifies around 39.34% of the wine samples. The results are comparable with the LDA model in part b. Showing that both models capture the same amount of structure in the data, the wine data overlaps and are not separable by a linear boundary.

2.4 (d)

```r
library(e1071)
```

```
## Warning: package 'e1071' was built under R version 4.4.3
```

```r
wine_nb <- naiveBayes(quality ~ ., data = Wine)
wine_nb_pred <- predict(wine_nb, newdata = Wine)
nb_class_err <- mean(wine_nb_pred != Wine$quality)
round(nb_class_err, 4)
```

```
## [1] 0.4396
```

The naive bayes classifier model has achieved a classification error of 0.4396, meaning it misclassifies around 43.96% of the wine samples. This is noticeably worse compared to the multinomial and LDA model, most likely the NB independent assumption is unrealistic for the wine dataset.

2.5 (e)

```
set.seed(4064)
K = 5
n= NROW(Wine)
idx = sample(n,n)
nk = n/K

nk = c(rep(320,4),319)
idx1 = cumsum(c(0,nk))
folds = list()
for(k in 1:K){
  folds[[k]] = idx[(idx1[k]+1):idx1[k+1]]
}

class_error_lda=0
class_error_multinom=0
class_error_nb = 0
for(k in 1:K){
  idx = folds[[k]]
  lda_model = lda(quality ~ ., data = Wine, subset = -idx)
  multinom_model = multinom(quality ~ ., data = Wine, subset = -idx, maxit = 1000)
  nb_model = naiveBayes(quality ~ ., data = Wine, subset = -idx)
  lda_pred = predict(lda_model, Wine[idx,])$class
  multinom_pred = predict(multinom_model, Wine[idx,])
  nb_pred = predict(nb_model, Wine[idx,])
  class_error_lda = class_error_lda + mean(lda_pred != Wine$quality[idx])
  class_error_multinom = class_error_multinom + mean(multinom_pred != Wine$quality[idx])
  class_error_nb = class_error_nb + mean(nb_pred != Wine$quality[idx])
}
```

```
## # weights:  78 (60 variable)
## initial  value 2291.660361
## iter  10 value 1588.496058
## iter  20 value 1433.145180
## iter  30 value 1204.661805
## iter  40 value 1163.426008
## iter  50 value 1157.705566
## iter  60 value 1155.995387
## iter  70 value 1155.169040
## iter  80 value 1154.340527
## iter  90 value 1153.910866
## iter 100 value 1153.714027
## iter 110 value 1153.107162
## iter 120 value 1152.825698
## iter 130 value 1152.803190
## iter 140 value 1152.801654
## iter 150 value 1152.777905
## iter 160 value 1152.656001
## iter 170 value 1152.338008
## iter 180 value 1152.080600
## iter 190 value 1151.784423
## iter 200 value 1151.553128
```

```
## iter 210 value 1151.228526
## iter 220 value 1150.778834
## iter 230 value 1149.483559
## iter 240 value 1147.649427
## final  value 1147.519383
## converged
## # weights:  78 (60 variable)
## initial  value 2291.660361
## iter  10 value 1540.209466
## iter  20 value 1397.741687
## iter  30 value 1213.789151
## iter  40 value 1183.459746
## iter  50 value 1176.082307
## iter  60 value 1174.826042
## iter  70 value 1173.686868
## iter  80 value 1172.984388
## iter  90 value 1172.765579
## iter 100 value 1172.630298
## iter 110 value 1172.448534
## iter 120 value 1172.384420
## final  value 1172.382879
## converged
## # weights:  78 (60 variable)
## initial  value 2291.660361
## iter  10 value 1519.425633
## iter  20 value 1380.291932
## iter  30 value 1200.406324
## iter  40 value 1175.311984
## iter  50 value 1168.658454
## iter  60 value 1166.513152
## iter  70 value 1165.073430
## iter  80 value 1163.389016
## iter  90 value 1163.135247
## iter 100 value 1162.896139
## iter 110 value 1162.595442
## iter 120 value 1162.380535
## final  value 1162.377748
## converged
## # weights:  78 (60 variable)
## initial  value 2291.660361
## iter  10 value 1504.324708
## iter  20 value 1368.801659
## iter  30 value 1205.905315
## iter  40 value 1178.313582
## iter  50 value 1171.559134
## iter  60 value 1169.939312
## iter  70 value 1168.952778
## iter  80 value 1168.100562
## iter  90 value 1167.541497
## iter 100 value 1167.299686
## iter 110 value 1167.232978
## iter 120 value 1165.661389
## iter 130 value 1165.259637
## iter 140 value 1165.250007
```

```
## iter 150 value 1165.158390
## iter 160 value 1164.570987
## iter 170 value 1164.430190
## iter 180 value 1164.340912
## iter 190 value 1164.266016
## iter 200 value 1164.248957
## iter 210 value 1164.120833
## iter 220 value 1163.858181
## iter 230 value 1163.152805
## iter 240 value 1162.796133
## final  value 1162.750231
## converged
## # weights:  78 (60 variable)
## initial  value 2293.452121
## iter  10 value 1582.827714
## iter  20 value 1457.176835
## iter  30 value 1218.867538
## iter  40 value 1182.981875
## iter  50 value 1177.134462
## iter  60 value 1175.691200
## iter  70 value 1173.926627
## iter  80 value 1173.143210
## iter  90 value 1172.994364
## iter 100 value 1172.737240
## iter 110 value 1172.586484
## iter 120 value 1172.152605
## iter 130 value 1172.125220
## iter 140 value 1172.123056
## iter 150 value 1172.082428
## iter 160 value 1171.919322
## iter 170 value 1171.742871
## iter 180 value 1171.621215
## iter 190 value 1171.408147
## iter 200 value 1171.264777
## iter 210 value 1170.928147
## iter 220 value 1170.395271
## iter 230 value 1169.536626
## iter 240 value 1167.890717
## final  value 1167.889227
## converged
```

```
round(class_error_lda/K, 4)
```

```
## [1] 0.4046
```

```
round(class_error_multinom/K, 4)
```

```
## [1] 0.4053
```

```
round(class_error_nb/K, 4)
```

```
## [1] 0.4559
```

Based on the 5 fold CV results, we can see that the LDA has the smallest misclassification CV error(0.4046), followed by the multinomial model(0.4053) and lastly the naive bayes model(0.4559). The results indicate that LDA and multinomial seem to handle the wine dataset pretty well given the low classification error. In contrast, naïve bayes has a higher classification error(0.4559) compared to the other two models. A possible reason may be that the dataset may have highly correlated variables, which leads to a strong violation of naïve bayes independence assumption. Overall, based on 5-fold CV, we can see that LDA provides the best out-of-sample predictive performance out of the 3 models.

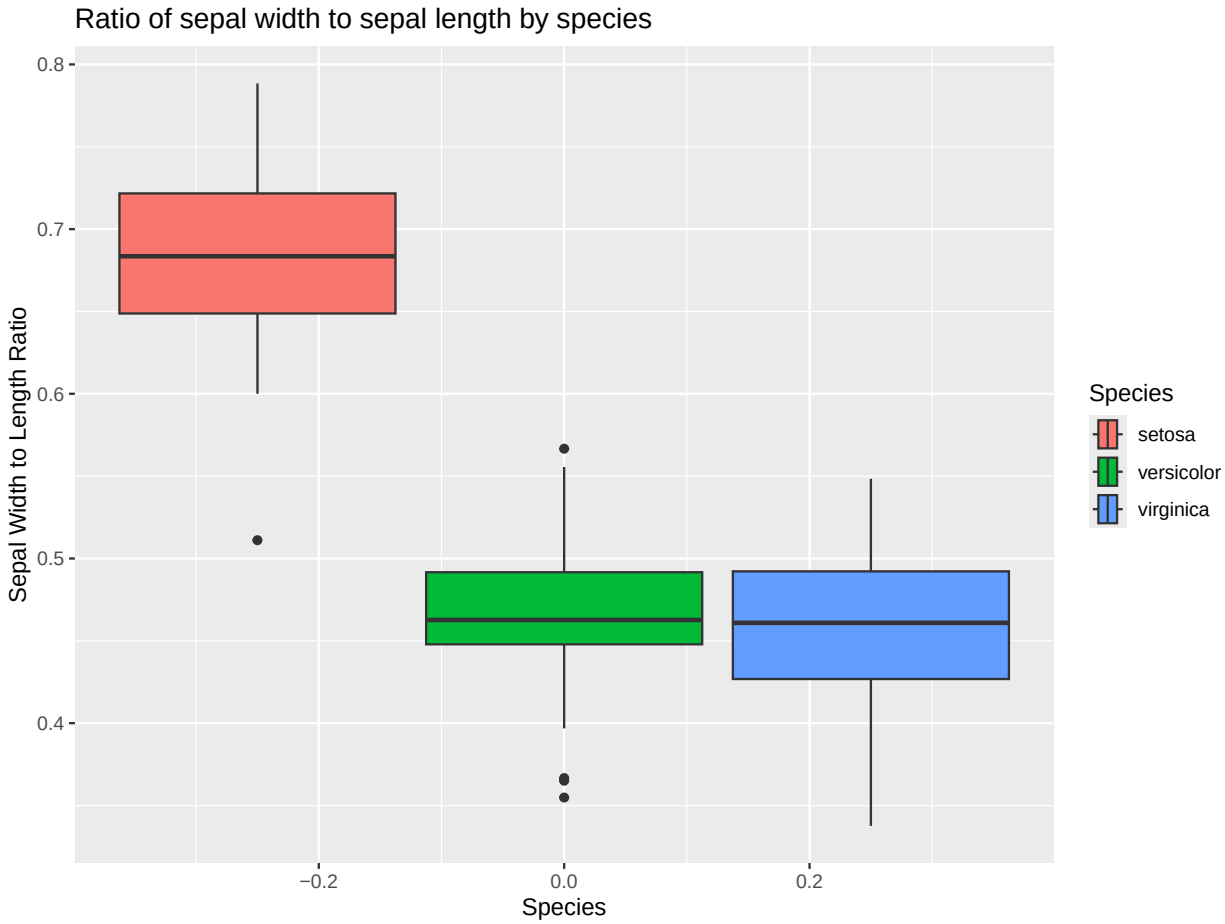
3 Question 3

3.1 (a)

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```
data(iris)
iris$r <- iris$Sepal.Width/iris$Sepal.Length
ggplot(iris, aes(x=r, fill =Species))+geom_boxplot()+coord_flip()+labs(title='Ratio of sepal width to s
```



The boxplot shows a clear difference in class distribution on the sepal width and sepal length ratio. We

can see that setosa has the highest median ratio, indicating the sepal width is relatively large compared to it's sepal length. While the remaining two classes have lower ratio. Versicolor shows a slightly higher ratio compared to virginica, but is a little lower spread compared to virginica, while virginica has a lower ratio and a bit higher spread. Moreover, the versicolor and virginica species are overlapping, suggesting these 2 are less distinct when it comes to sepal width and sepal length ratio compared to the setosa species.

3.2 (b)

```
r_med_hat <- median(iris$r[iris$Species == 'setosa'])
round(r_med_hat, 4)
```

```
## [1] 0.6835
```

The median sepal width by sepal length ratio for setosa species is 0.6835, this means the setosa species have a higher sepal width to sepal length median ratio compared to the other species. This is inline with what can be seen in the boxplot in part a.

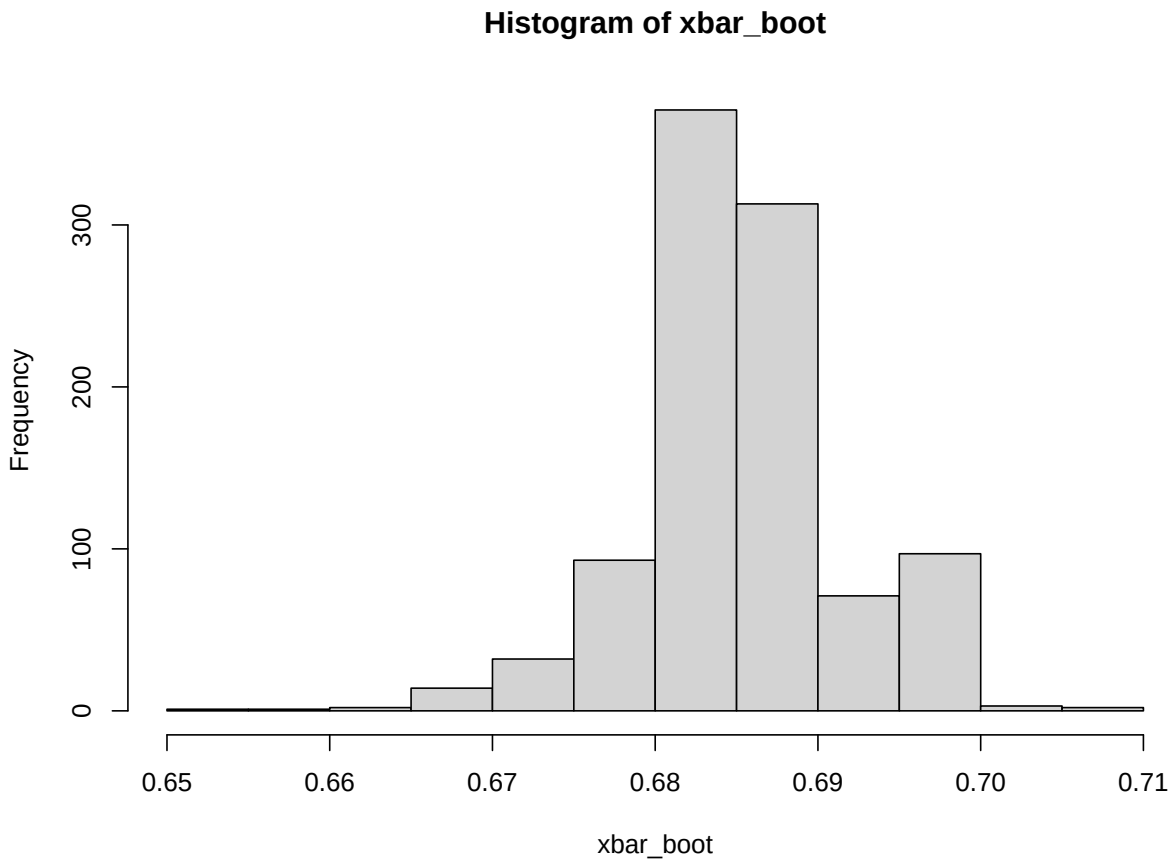
3.3 (c)

```
set.seed(4064)
nBoot <- 1000
xbar_boot <- rep(NA, nBoot)
setosa_r <- iris$r[iris$Species == 'setosa']
for(b in 1:nBoot){
  # resample the data
  setosa_r_boot <- sample(setosa_r, length(setosa_r), replace = TRUE)
  # compute sample mean of the resample data
  xbar_boot[b] <- median(setosa_r_boot)
}
sd_r <- round(sd(xbar_boot),4)
sd_r
```

```
## [1] 0.0065
```

The boostrap standard error is approximately 0.0065, suggesting that the sample median of the sepal width to length ratio for setosa is fairly stable with fairly low spread.

```
hist(xbar_boot)
```



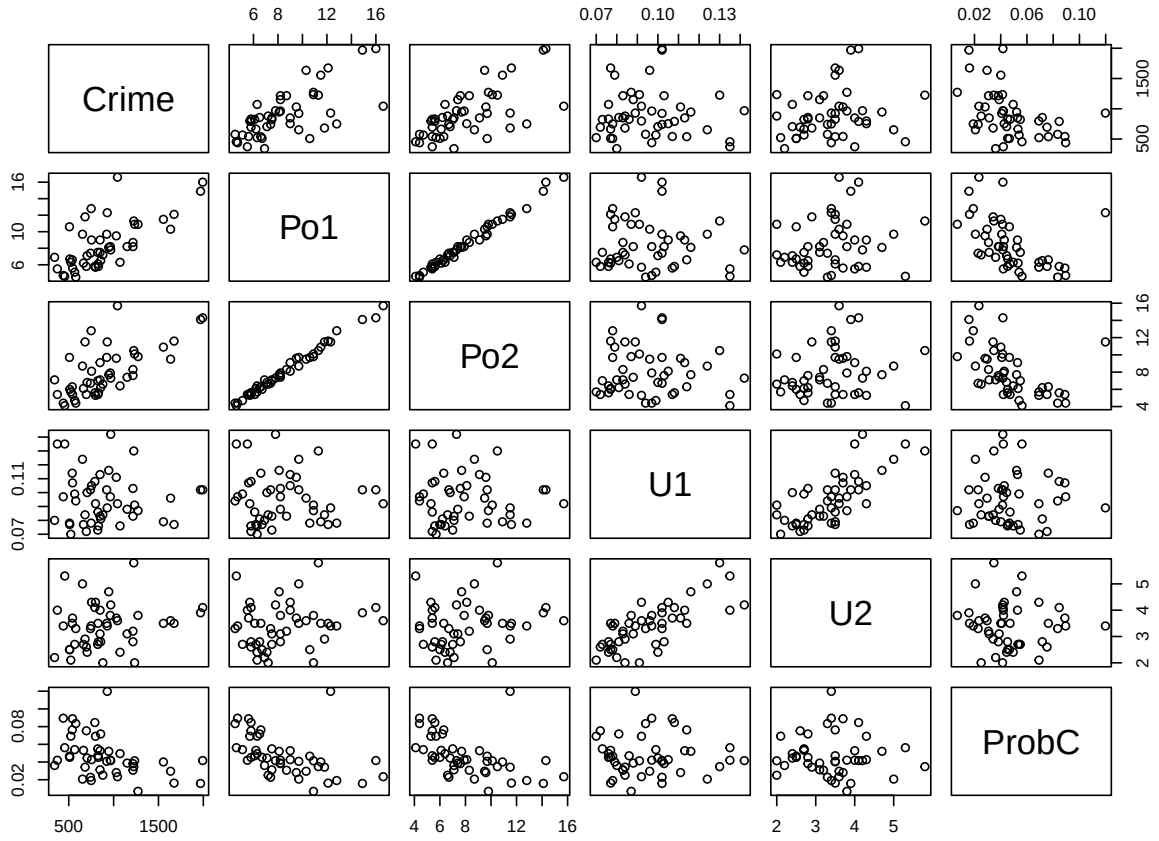
The bootstrap of distribution of the setosa median ratio is near bell-shaped and seems to be centered between 0.68 and 0.69, which is consistent with the median ratio found in part b(0.6835). The bootstrap results show that the median sepal width to length ratio for setosa is fairly stable and well estimated.

4 Question 4

4.1 (a)

Here we read in the USCrime dataset and plot the pairs plot for each variable.

```
UScrime <- read.table('UScrime.txt', header = TRUE)
pairs(~ Crime + Po1 + Po2 + U1 + U2 + ProbC, data = UScrime)
```



4.2 (b)

Yes, we can remove Po2 and U1. As seen above in part a's pairs plot, Po1 and Po2 are almost perfectly linearly related, if we keep both, we're basically adding redundant information and inflate the regression model's variance. Similarly, for U1 and U2, it's also shown a linear relationship between the 2, suggesting it does not contribute much additional information that U2 can already provide. Thus, we can opt to remove Po2 and U1.

4.3 (c)

We first remove the 2 variables, Po2 and U1, and we double check it by populating the data frame. We can see Po2 and U1 have been successfully removed.

```
reduced_USCrime <- UScrime[, !names(UScrime)%in%c('Po2', 'U1')]
str(reduced_USCrime)
```

```
## 'data.frame':    47 obs. of  14 variables:
##  $ M     : num  15.1 14.3 14.2 13.6 14.1 12.1 12.7 13.1 15.7 14 ...
##  $ So    : int  1 0 1 0 0 0 1 1 1 0 ...
##  $ Ed    : num  9.1 11.3 8.9 12.1 12.1 11 11.1 10.9 9 11.8 ...
##  $ Po1   : num  5.8 10.3 4.5 14.9 10.9 11.8 8.2 11.5 6.5 7.1 ...
```

```
##  $ LF    : num  0.51 0.583 0.533 0.577 0.591 0.547 0.519 0.542 0.553 0.632 ...
##  $ MF    : num  95 101.2 96.9 99.4 98.5 ...
##  $ Pop   : int  33 13 18 157 18 25 4 50 39 7 ...
##  $ NW    : num  30.1 10.2 21.9 8 3 4.4 13.9 17.9 28.6 1.5 ...
##  $ U2    : num  4.1 3.6 3.3 3.9 2 2.9 3.8 3.5 2.8 2.4 ...
##  $ Wealth: int  3940 5570 3180 6730 5780 6890 6200 4720 4210 5260 ...
##  $ Ineq  : num  26.1 19.4 25 16.7 17.4 12.6 16.8 20.6 23.9 17.4 ...
##  $ ProbC : num  0.0846 0.0296 0.0834 0.0158 0.0414 ...
##  $ Time  : num  26.2 25.3 24.3 29.9 21.3 ...
##  $ Crime : int  791 1635 578 1969 1234 682 963 1555 856 705 ...
```

We then import the leaps library which contains the regsubsets() function, which we will need to apply the forward stepwise function.

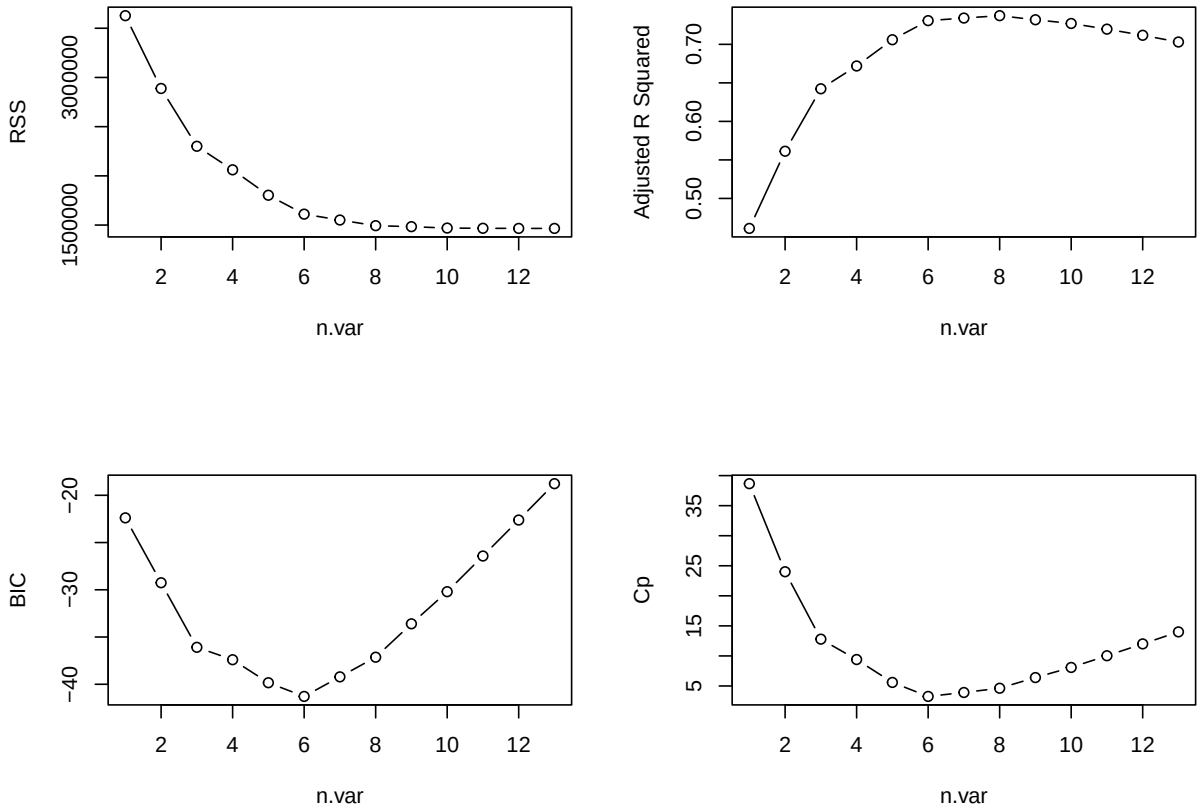
```
library(leaps)
```

```
## Warning: package 'leaps' was built under R version 4.4.3
```

```
reg_fwd <- regsubsets(Crime~., data = reduced_USCrime, method='forward', nvmax=13)
reg_fwd.summary <- summary(reg_fwd)
```

After running the forward stepwise function we plot the RSS, Adjusted R Square, BIC and Mallow's Cp against the number of variables, to visualize the minimum number of variables per metric.

```
par(mfrow=c(2,2))
plot(1:13, reg_fwd.summary$rss, xlab = 'n.var', ylab = 'RSS', type = 'b')
plot(1:13, reg_fwd.summary$adjr2, xlab = 'n.var', ylab = 'Adjusted R Squared', type = 'b')
plot(1:13, reg_fwd.summary$bic, xlab = 'n.var', ylab = 'BIC', type = 'b')
plot(1:13, reg_fwd.summary$cp, xlab = 'n.var', ylab = 'Cp', type = 'b')
```



The plots show that for RSS, as the number of variables increase, RSS decreases, this is expected since adding more variables always reduces the residual errors. But we can see that by variable 6, it does plateau off. Overall, it shows that the most optimal number of variables is using all 13 variables. But, RSS alone is not enough to choose the best model, since the residual errors always decreases when we add more variables, thus it always wants a model with many predictors. The adjusted R square shows as the number of variable increase, adjusted R square increases. We can see that as more variables were added the adjusted r square up, but by the 8th variable, it starts to decline. Overall, adjusted r square suggests that only 8 variables are needed for the most optimal model. The BIC shows that as it increases it drops to the minimum at 6 variables. This means the optimal number of variables to have is 6 based on BIC. The Cp shows that as it increases it drops to the minimum at 6 variables. This means the optimal number of variables to have is 6 based on Cp.

```
(rss_var <- which.min(reg_fwd.summary$rss))
```

```
## [1] 13
```

```
(adjr2_var <- which.max(reg_fwd.summary$adjr2))
```

```
## [1] 8
```

```
(bic_var <- which.min(reg_fwd.summary$bic))
```

```
## [1] 6
```

```
(cp_var <- which.min(reg_fwd.summary$cp))
```

```
## [1] 6
```

RSS shows that the best model with a good balance of fit and complexity must have 13 variables, for Adjusted R square it is 8, and for BIC and Cp it is 6 variables. RSS alone is not enough to choose the best model, since the residual errors always decreases when we add more variables.

```
round(coef(reg_fwd, rss_var), 4)
```

```
## (Intercept)           M          So          Ed         Po1          LF
##  -5781.4546     93.9001     77.6600    161.1992    108.7879    495.5981
##          MF         Pop          NW          U2      Wealth        Ineq
##      4.1693     -1.0745      1.0884     86.5139      0.1028     74.4182
##       ProbC        Time
##  -4262.3744     -0.2103
```

For RSS the optimal model contains the following variables: M, So, Ed, Po1, LF, MF, Pop, NW, U2, Wealth, Ineq, ProbC, Time.

The optimal model size based on RSS is written as: {

$$\hat{Crime} = -5781.4546 + 93.9001 \times M + 77.6600 \times So + 161.1992 \times Ed + 108.7879 \times Po1 + 495.5981 \times LF + 4.1693 \times MF - 1.0745 \times Pop + 1.0884 \times$$

}

```
round(coef(reg_fwd, adjr2_var), 4)
```

```
## (Intercept)           M          Ed         Po1         Pop          U2
##  -5603.0868    104.2903    168.3945    112.5287     -1.2453     84.1295
##      Wealth        Ineq       ProbC
##      0.1191     84.5217  -3786.1047
```

For adjusted r square the optimal model contains the following variables: M, Ed, Po1, Pop, U2, Wealth, Ineq, ProbC.

The optimal model size based on Adjusted R Square is written as:

$$\hat{Crime} = -5603.0868 + 104.2903 \times M + 168.3945 \times Ed + 112.5287 \times Po1 - 1.2453 \times Pop + 84.1295 \times U2 + 0.1191 \times Wealth + 84.5217 \times Ineq - 378$$

```
round(coef(reg_fwd, bic_var), 4)
```

```
## (Intercept)           M          Ed         Po1          U2        Ineq
##  -5040.5050    105.0196    196.4712    115.0242     89.3660     67.6532
##       ProbC
##  -3801.8363
```

For Bic the optimal model contains the following variables: M, Ed, Po1, U2, Ineq, ProbC.

The optimal model size based on BIC is written as:

$$\hat{Crime} = -5040.5050 + 105.0196 \times M + 196.4712 \times Ed + 115.0242 \times Po1 + 89.3660 \times U2 + 67.6532 \times Ineq - 3801.8363 \times ProbC$$

```
round(coef(reg_fwd, cp_var), 4)
```

```
## (Intercept)           M          Ed         Po1          U2         Ineq
##  -5040.5050    105.0196    196.4712    115.0242     89.3660     67.6532
##       ProbC
##  -3801.8363
```

For Cp the optimal model contains the following variables: M, Ed, Po1, U2, Ineq, ProbC

The optimal model size based on Cp is written as:

$\hat{Crime} = -5040.5050 + 105.0196 \times M + 196.4712 \times Ed + 115.0242 \times Po1 + 89.3660 \times U2 + 67.6532 \times Ineq - 3801.8363 \times ProbC$

4.4 (d)

We first create the code to perform 5 fold cv for the best-subset regression, computes the test MSE for the reducedUSCrime dataset and returns the cv error for each model so we can choose the best number of predictors. i.)

```
K <- 5
n <- nrow(reduced_USCrime)
set.seed(40643406)
folds <- sample(rep(1:K, length = n))
cv.errors <- matrix(NA, K, 13)
```

ii.)

```
for(k in 1:K){
  id <- which(folds == k)
  best.fit <- regsubsets(Crime~., data = reduced_USCrime[-id,], nvmax = 13)

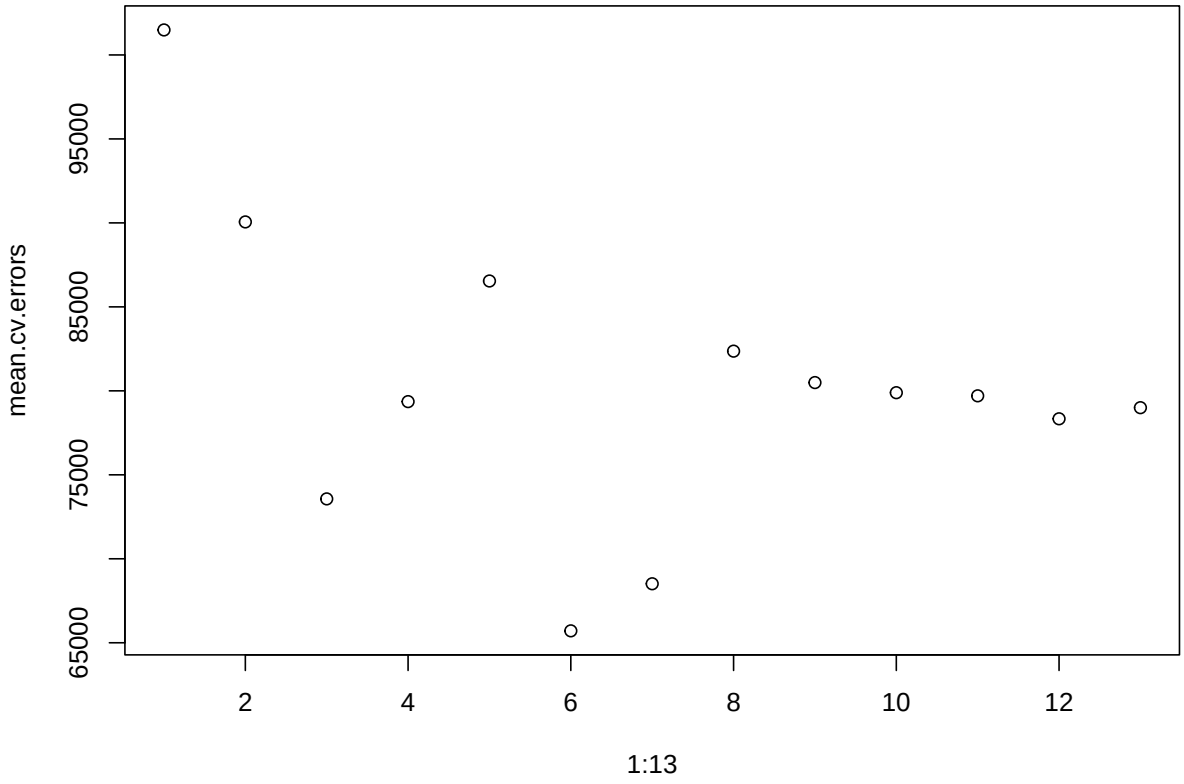
  for(i in 1:13){
    coefi <- coef(best.fit, id = i)
    xvars <- names(coefi)

    form <- as.formula(best.fit$call[[2]])
    mat <- model.matrix(form, data = reduced_USCrime[id,])
    pred.tmp <- mat[, xvars] %*% coefi
    cv.errors[k, i] <- (length(id) / n) * mean((reduced_USCrime$Crime[id] - as.vector(pred.tmp))^2)
  }
}
(mean.cv.errors <- colSums(cv.errors))
```

```
##  [1] 101488.24  90057.43  73566.53  79358.06  86540.79  65709.89  68512.47
##  [8]  82362.88  80492.22  79889.85  79702.79  78335.20  78997.81
```

After running the 5 fold cv, we then plot them to see which number of variables give the lowest prediction error is 6 (65709.89).

```
plot(1:13, mean.cv.errors)
```



The plot supports our findings, as the number of variables reached 6, it has reached the minimum prediction error. Thus, we can conclude that the best model with a balanced of fit and complexity must have 6 variables.

4.5 (e)

To get the variables, we first run the best fit model then we get the minimum prediction errors, which is 6.

```
regfit.best <- regsubsets(Crime~., data=reduced_USCrime, nvmax=13)
(minimum_var <- which.min(mean.cv.errors))
```

```
## [1] 6
```

We then get the 6 variables

```
round(coef(regfit.best, minimum_var), 4)
```

```
## (Intercept)           M          Ed         Po1          U2        Ineq
##  -5040.5050    105.0196    196.4712    115.0242     89.3660     67.6532
##       ProbC
##  -3801.8363
```

The model should have these 6 variables to provide the best balance between complexity and predictive accuracy: M, Ed, Po1, U2, Ineq, ProbC

We write the optimal model as:

$\hat{Crime} = -5040.5050 + 105.0196 \times M + 196.4712 \times Ed + 115.0242 \times Po1 + 89.3660 \times U2 + 67.6532 \times Ineq - 3801.8363 \times ProbC$